

asteesia_drone_sim

ROS2 Humble 四旋翼无人机仿真器

中山大学 2027 保研招生程序设计题 (60 分)

July 22, 2026

技术报告

本报告系统阐述四旋翼无人机仿真器的架构设计、动力学建模、控制器推导、四模式感知/避障规划、可视化方案和实验结果。所有源代码与文档开源在

github.com/bestastesia/astesia_drone_sim/tree/v1.1.0

本报告对应最新的 v1.1.0 分支 (不是 main)。

演示视频共 10 段, 覆盖全部验收场景及 bonus 测试。

GitHub 演示视频

10 段演示视频均已随报告提交到 GitHub:

[report/videos/](#)

文件名与第 6 节场景一一对应; 本地录制目录仅作为原始素材来源。

Contents

1	系统架构	3
2	动力学模型	4
3	控制器设计	5
4	地图与避障设计	6
5	可视化方案	7
6	实验结果	8
7	与参考仓库的关系	9
8	AI 使用说明	9
9	反思：如果继续做两周	10

1 系统架构

1.1 工程结构

仿真器由 6 个 ROS2 ament_cmake 包组成，全 C++17 + rclcpp 实现。编译器 g++ 11+，构建系统 colcon，消息全部使用 ROS2 标准接口。

Table 1: 包结构及职责

包名	职责
drone_common	header-only 共享库：坐标系、控制分配矩阵 B/B^{-1} 、四元数积分、障碍物几何体、体素栅格、传感器噪声模型
drone_dynamics	1 kHz 动力学积分：平动 + 转动 EOM、电机一阶滞后、风扰、IMU/Odom 噪声
drone_controller	200 Hz 级联控制器：线性 PD + 死区积分 + 反饱和位置环、Lee 几何姿态环、mixer 逆、LADRC（实验性）
drone_map	1 Hz 障碍物发布：显式 YAML / 程序化随机生成（固定 seed 可复现）
drone_planner	1 Hz 四模式路径规划：Global/FOV × 2D/3D、A* (8 连通) / A*3D (26 连通)、FOV 可视化
drone_bringup	启动文件、RViz 配置、地面站 (live_monitor.py)、辅助脚本、报告

1.2 节点拓扑与数据流

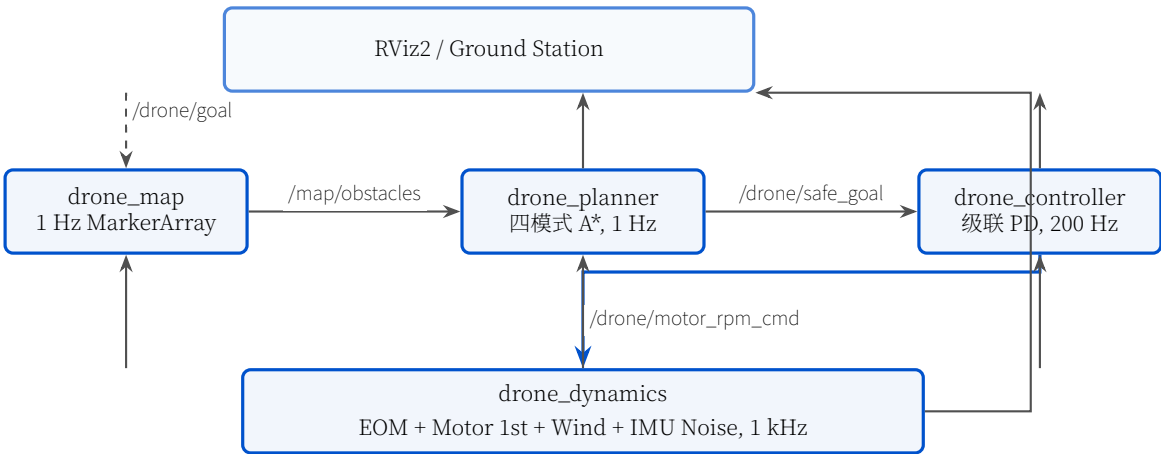


Figure 1: 系统节点拓扑与 ROS2 话题数据流。实线 = 核心闭环；虚线 = 用户交互

Table 2: 全部 ROS2 话题一览

Topic	类型	方向	说明
/drone/motor_rpm_cmd	Float32MultiArray[4]	ctrl → dyn	电机 RPM [FL,FR,BL,BR]
/drone/odom	Odometry	dyn → ctrl,planner	里程计 (100Hz)
/drone/imu	Imu	dyn → rviz	IMU 含噪测量 (100Hz)
/drone/path	Path	dyn → rviz	历史飞行轨迹
/drone/goal	PoseStamped	user/rviz → planner	用户目标点
/drone/safe_goal	PoseStamped	planner → ctrl	避障后安全子目标
/drone/planned_path	Path	planner → rviz	A* 规划路径
/drone/planner_status	String	planner → rviz,GS	规划器状态
/drone/fov_cone	Marker (LINE_LIST)	planner → rviz	FOV 感知锥体
/drone/fov_voxels	MarkerArray (CUBE_LIST)	planner → rviz	FOV 可见体素
/map/obstacles	MarkerArray	map → planner,rviz	障碍物几何体
/drone/drone_marker	MarkerArray	dyn → rviz	无人机 ARROW
/tf	TFMessage	dyn → rviz	tf 变换树

1.3 tf 树

- map (fixed frame)
 - └─ base_link (dynamics broadcaster, 100Hz)
 - └─ imu_link (static, identity)

1.4 坐标系约定 (REP-103 ENU)

世界系 map: +x 前/东, +y 左/北, +z 上, 重力沿 $-z$ 。

机体系 base_link: +x 前, +y 左, +z 上, 右手系。

X 型四旋翼: 臂长 $l = 0.2\text{m}$ 。机体系电机坐标 FL($+l/\sqrt{2}, +l/\sqrt{2}$), FR($+l/\sqrt{2}, -l/\sqrt{2}$), BL($-l/\sqrt{2}, +l/\sqrt{2}$), BR($-l/\sqrt{2}, -l/\sqrt{2}$)。对角同向 (PX4 风格): FL/BR 为 CCW(+), FR/BL 为 CW(-)。电机索引 [FL, FR, BL, BR] 通过 MotorIndex 枚举全局绑定, 禁止各节点硬编码。

2 动力学模型

2.1 状态量与输入

$$\mathbf{x} = [\mathbf{p}_W, \mathbf{v}_W, \mathbf{q}_{WB}, \boldsymbol{\omega}_B]^\top \in \mathbb{R}^3 \times \mathbb{R}^3 \times S^3 \times \mathbb{R}^3$$

电机转速: $\omega_i \in [\omega_{\min} = 0, \omega_{\max} = 1000 \text{ rad/s}]$, $i \in \{\text{FL, FR, BL, BR}\}$

$\mathbf{p}_W$ = 世界系位置, $\mathbf{v}_W$ = 世界系速度, $\mathbf{q}_{WB}$ = 姿态四元数 (机体系 $\rightarrow$ 世界), $\boldsymbol{\omega}_B$ = 机体系角速度。转动惯量 $\mathbf{I} = \text{diag}(0.02, 0.02, 0.04) \text{ kg}\cdot\text{m}^2$ 。

2.2 推力与力矩

单电机: 推力 $F_i = k_F \omega_i^2$ (沿机体 $+z$), 反扭矩 $M_{z,i} = k_M \cdot \text{sgn}_i \cdot \omega_i^2$ 。四个电机的合成力与力矩通过控制分配矩阵 $\mathbf{B} \in \mathbb{R}^{4 \times 4}$ 计算:

$$[F, \tau_x, \tau_y, \tau_z]^\top = \mathbf{B} \cdot [\omega_1^2, \omega_2^2, \omega_3^2, \omega_4^2]^\top, \quad \mathbf{B} = \begin{bmatrix} k_F & k_F & k_F & k_F \\ +a & -a & +a & -a \\ -a & -a & +a & +a \\ +k_M & -k_M & -k_M & +k_M \end{bmatrix} \quad (1)$$

$a = l/\sqrt{2} \cdot k_F$ 。各行物理意义: τ_x 来自推力臂对 roll 的叉乘 ($\sum r_{y,i} F_i$), τ_y 来自对 pitch 的贡献 ($-\sum r_{x,i} F_i$), τ_z 来自反扭矩。B 非奇异 (X 配置), 控制器预计算 $\mathbf{B}^{-1}$ 。

2.3 运动方程 (EOM)

平动 (世界系):

$$\ddot{\mathbf{p}}_W = \frac{F}{m} \mathbf{R}_{WB}(\mathbf{q}) \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} + \frac{\mathbf{F}_{\text{wind}}}{m} \quad (2)$$

$\mathbf{R}_{WB}(\mathbf{q}) \in SO(3)$ 为四元数对应的旋转矩阵, $g = 9.80665 \text{ m/s}^2$ 。

转动 (机体系):

$$\dot{\boldsymbol{\omega}}_B = \mathbf{I}^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega}_B \times (\mathbf{I} \boldsymbol{\omega}_B)) \quad (3)$$

电机一阶: $\dot{\omega}_i = (\omega_{\text{cmd},i} - \omega_i)/\tau_m$, $\tau_m = 0.02\text{s}$ 。

2.4 积分方案

半隐式 Euler, $\Delta t = 1 \text{ ms}$:

$$\begin{aligned} \mathbf{v}_W &+= \mathbf{a}_W \cdot \Delta t \\ \mathbf{p}_W &+= \mathbf{v}_W \cdot \Delta t \\ \boldsymbol{\omega}_B &+= \dot{\boldsymbol{\omega}}_B \cdot \Delta t \\ \mathbf{q}_{WB} &\leftarrow \text{normalize}(\mathbf{q}_{WB} + \frac{\Delta t}{2} (\mathbf{q}_{WB} \otimes [0, \boldsymbol{\omega}_B])) \end{aligned} \quad (4)$$

每步四元数归一化。1 kHz 步长下显式积分足够稳定。

2.5 风扰与传感器噪声

风扰: $F_{\text{wind}} = F_{\text{const}} + F_{\text{gust}}$ 。阵风 = 正弦调制 $A \sin(2\pi t/T + \phi)$ + Ornstein-Uhlenbeck 湍流 ($\theta = 1\text{s}$ 均值回复率)。

IMU 噪声: 偏置 b_a, b_g + 白噪声 $\mathcal{N}(0, \sigma^2)$ + 偏置随机游走:

$$\tilde{a} = a_{\text{true}} + b_a + \mathcal{N}(0, \sigma_a^2), \quad \dot{b}_a = \mathcal{N}(0, \sigma_{a,\text{rw}}^2) \quad (5)$$

Odom 位置/速度各加独立高斯噪声 $\sigma_{\text{pos}}, \sigma_{\text{vel}}$ 。tf 始终用 ground truth。

3 控制器设计

3.1 控制架构

级联结构: 外环位置 PD + 积分 $\rightarrow$ 期望加速度 $\rightarrow$ 推力分解 $\rightarrow$ Lee 几何姿态误差 $\rightarrow$ 力矩命令 $\rightarrow$ mixer 逆 $\rightarrow$ 4 RPM。控制频率 200 Hz, $\Delta t_{\text{ctrl}} = 5\text{ ms}$ 。

3.2 外环: 线性 PD + 有限积分 + 反饱和

hover/waypoint-hold 模式 ($v_{\text{des}} = 0$):

$$e_p = p_{\text{goal}} - p \quad (6)$$

$$e_v = -v \quad (7)$$

$$a_{\text{des}} = K_p \circ e_p + K_d \circ e_v + a_I \quad (8)$$

$\circ$ = 逐元素乘法。增益按 $\zeta = K_d/(2\sqrt{K_p}) \approx 0.7$:

$$K_p = [2.0, 2.0, 3.0]^\top, \quad K_d = [2.0, 2.0, 2.4]^\top \quad (9)$$

积分项 (V2 设计): 0.02 m 死区——小误差时 $\times 0.99$ /步衰减; conditional anti-windup——仅当 PD + I 不超限幅时累加; 积分限幅 $K_{i,\text{max}} = 2.0\text{ m/s}^2$ 。仅 x 轴启用 ($K_i = 0.5$) 补偿稳态风扰。

加速度限幅: $a_{xy} \leq \pm 4\text{ m/s}^2$, $a_z \in [-3, 6]\text{ m/s}^2$, $\|a\|_{\text{vec}} \leq 8\text{ m/s}^2$ 。远目标 ($\|e_p\| > 10\text{ m}$) 钳位到 a_{max} 方向并清零积分。

3.3 内环: Lee 几何姿态误差 (无 Euler 奇异)

$$T_W = m(a_{\text{des}} + g \hat{z}_W) \quad (10)$$

$$F = \|T_W\| \quad (11)$$

$$\hat{z}_{B,\text{des}} = T_W / F \quad (12)$$

由 $\hat{z}_{B,\text{des}} + \psi_{\text{des}}$ 构造 R_{des} (世界系投影法, 永不超过 gimbal lock):

$$x_C = [\cos \psi_{\text{des}}, \sin \psi_{\text{des}}, 0]^\top, \quad \hat{y}_B = \frac{\hat{z}_B \times x_C}{\|\hat{z}_B \times x_C\|}, \quad \hat{x}_B = \hat{y}_B \times \hat{z}_B \quad (13)$$

$$R_{\text{des}} = [\hat{x}_B \mid \hat{y}_B \mid \hat{z}_B] \quad (14)$$

几何姿态误差 (Lee et al., 2010):

$$R_{\text{err}} = R_{\text{des}}^\top R - R^\top R_{\text{des}}, \quad e_R = \frac{1}{2} \text{vee}(R_{\text{err}}) \quad (15)$$

$\text{vee}(\cdot)$ 将反对称阵映射到 $\mathbb{R}^3$ 。

$$\tau_{\text{des}} = -K_{p,R} \circ e_R - K_{d,\omega} \circ \omega_B \quad (16)$$

$K_{p,R} = [8, 8, 3]^\top$, $K_{d,\omega} = [0.8, 0.8, 0.7]^\top$ 。

3.4 Mixer 逆与饱和链

完整限幅链: $a_{\text{des}} \text{ clamp} \rightarrow \text{far-goal failsafe} \rightarrow F \text{ clamp}_{[F_{\min}, F_{\max}]} \rightarrow \tau \text{ clamp}_{[-\tau_{\max}, \tau_{\max}]} \rightarrow B^{-1}$:

$$[\omega_1^2, \dots, \omega_4^2]^\top = B^{-1} \cdot [F, \tau_x, \tau_y, \tau_z]^\top \quad (17)$$

$\omega_i^2 \text{ clamp } [0, \omega_{\max}^2]$ (负值取 0) $\rightarrow$ RPM clamp $\rightarrow$ 发布。低速大 yaw 时个别电机停转导致 yaw authority 下降 (README 注明)。

4 地图与避障设计

4.1 障碍物表示与地图生成

几何体: 轴对齐球 (radius r)、圆柱 (radius r , height h)、立方体 (half_extents $[h_x, h_y, h_z]$), 定义于 drone_common/obstacle.hpp。

地图模式: drone_map 支持 (1) 显式 YAML 列表——字符串格式 "shape cx cy cz params..."; (2) 程序化随机——固定 seed 在 bounds 内采样, 拒绝靠近 start/goal 和重叠者。同一 seed 完全可复现。

4.2 碰撞检测与安全距离

2D: 膨胀半径 = safety_distance (0.4 m) + drone_radius (0.2 m) = 0.6 m。sphere/cylinder $\rightarrow$ 圆盘 (距离检查); cube $\rightarrow$ AABB (整格标记)。 z_{cruise} 跨度检查——障碍物不含巡航高度的忽略。

3D: 体素栅格 resolution 0.2 m, 各形状标记占用体素, 膨胀半径同样施加。

4.3 四模式规划

Table 3: 四模式感知/规划矩阵

全局 (GLOBAL)	局部 (FOV)
2D A* 模式 1 (默认): 全图 8 连通 A*, octile 启发	模式 2: FOV 可见体素 $\rightarrow$ 2D 栅格投影, 不可见 = 未知
3D A* 模式 3: 全图 26 连通 3D A*	模式 4: FOV 内 3D A*, 不可见 = 占用 (保守)

2D A* (8 连通) octile 启发 $h = \max(\Delta x, \Delta y) + (\sqrt{2} - 1) \min(\Delta x, \Delta y)$ 。Bresenham 直线碰撞检查平滑——只跳掉安全直线段; 平滑坍塌时保留原始路径。

3D A* (26 连通) 3D octile 启发 $h = d_{\max} + (\sqrt{2} - 1)d_{\text{mid}} + (\sqrt{3} - \sqrt{2})d_{\min}$ 。3D Bresenham 平滑 + 坍塌保护。BFS 最近自由格 (半径 20) 处理 start/goal 在占用格中。

FOV 感知 水平/垂直视场角 θ_h (默认 60°), θ_v (默认 45°), 感知距离 R (默认 5 m)。每体素计算与机体 $+x$ 夹角判断锥体内。

safe_goal 推进 每 replan 周期 (1 Hz) 重规划整条路径, 但 safe_goal 最多前进 1 航点/周期——防止路径跳变引发控制器震荡。到达终点时 status = AT_GOAL。

失败处理: NO_PATH $\rightarrow$ 保留 last safe_goal, 悬停; NO_ODOM $\rightarrow$ 不规划; WAITING_FOR_MAP $\rightarrow$ 原 goal 作为 safe_goal。

5 可视化方案

5.1 RViz2 预设

Table 4: RViz2 显示配置

Display	Topic	视觉	含义
Drone Marker	/drone/drone_marker	绿箭头	位置 + 朝向
Odometry	/drone/odom	ODO 文字	位姿 + 速度
Path (history)	/drone/path	绿线	历史轨迹
Path (planned)	/drone/planned_path	黄线	A* 规划路径
Obstacles	/map/obstacles	橙色半透明	障碍物几何体
Goal	/drone/goal	红点	用户目标
Safe Goal	/drone/safe_goal	蓝点	安全子目标
FOV Cone	/drone/fov_cone	青色线框	FOV 锥体
FOV Voxels	/drone/fov_voxels	绿色方块	可见占用体素
TF	坐标轴 (红 = +x, 绿 = +y, 蓝 = +z)	—	—

5.2 地面站 (Web Dashboard)

live_monitor.py 内置 HTTP 服务器 (端口 8765) + rclpy 订阅, Chart.js 渲染 4 图表。零额外依赖。

1. Position Error $|e_x|$ (绿), $|e_z|$ (蓝), 标称线 0.3 m
2. Motor RPM FL/FR/BL/BR 四轴曲线, 自动缩放
3. XY Trajectory 轨迹 (绿) + 障碍物 (彩色圆) + goal (蓝点) + safe_goal (黄点)
4. Obstacle Distances 各障碍物距离 (彩色) + 最近距离 (绿粗线) + 安全线 (红虚线)

控制面板: 目标点下发、感知模式切换 (Global/FOV, 2D/3D)、FOV 参数滑块 (H/V/Range)、控制器热调参 (8 滑块)、风扰/噪声开关与预设 (Light/Medium/Strong)、紧急停机。Ctrl+C 退出时自动保存 dashboard_log.csv。

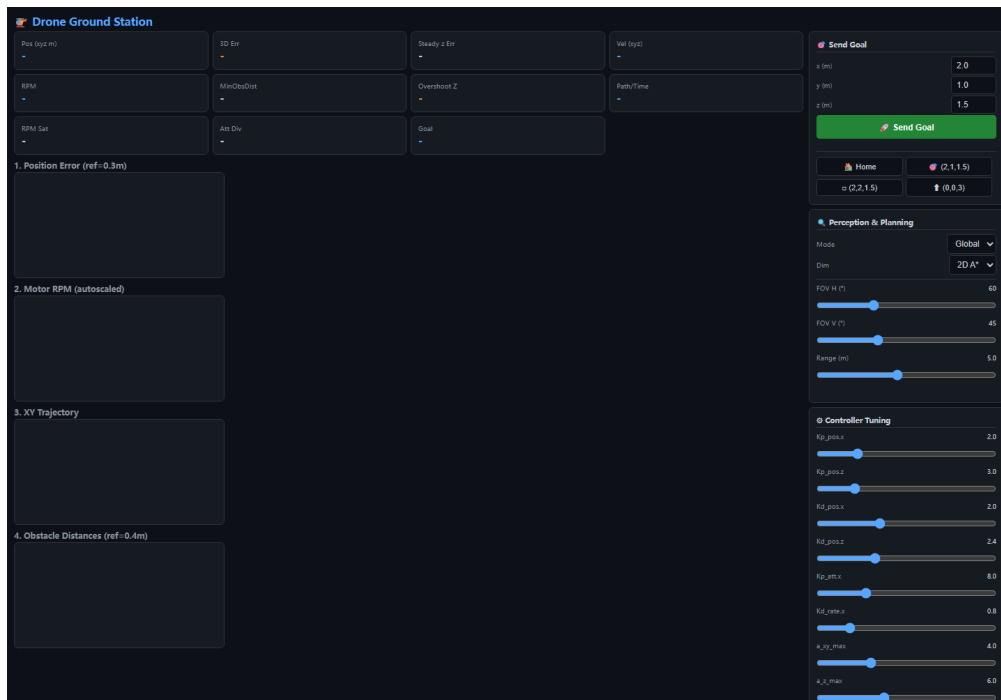


Figure 2: 地面站 Web Dashboard 实际界面截图 (v1.1.0)。

6 实验结果

6.1 验收场景总览

Table 5: 6 个验收测试及 bonus 场景

#	场景	结果	GitHub 视 频 文 件 (v1.1.0/report/videos)
1	悬停 (0,0,1.5), 8s	✓	1 初始高度改变的悬停.mp4
2	定点飞行 (2,1,1.5)	✓	2 定点飞行.mp4
3	正方形航线 4 点	✓	3 无规划器走正方形轨迹.mp4 3 带规划器走正方形轨迹.mp4
4	静态避障 (6+ 障碍物)	✓	4 静态绕行 2 维规划器.mp4 4 静态绕行 3 维规划器.mp4
5	狭窄通道绕行	✓	5 显式设计绕行 2 维与 3 维规划器.mp4
6	风扰模拟恢复 (4 级风)	✓	6 不同等级风扰模拟恢复演示.mp4
B1	IMU 干扰下规划器恢复	✓	7 强传感器 imu 干扰且无滤波
下规划器恢复情况.mp4			
B2	任意轨迹跟踪 (8 形)	✓	8 任意轨迹跟踪 8 形环绕演示.mp4

视频链接目录：github.com/.../tree/v1.1.0/report/videos。表中列出的文件名可直接在该目录打开。

6.2 关键性能指标

Table 6: 控制器关键性能指标

指标	说明	数值
稳态位置误差 (3σ)	悬停 15s, 最后 20% 数据	$< 0.02\text{ m}$
z 轴最大超调	$z_{\max} - 1.5$ (悬停)	$< 0.02\text{ m}$
正方形航线到达率	4 点, tolerance $< 0.3\text{m}$	4/4
最小障碍物距离	膨胀后 (0.6 m)	$> 0.6\text{ m}$
RPM 悬停稳态	hover 1.5m, 4 电机均值	[14.9, 15.1]
规划器重规划	replan_rate	1 Hz
控制器频率	ctrl_dt	200 Hz (5 ms)
动力学步长	sim_dt	1 kHz (1 ms)

6.3 可复现实测曲线

为避免只报告预览视频的结论，使用本分支在 Ubuntu 24.04/ROS 2 Jazzy 下重新录制了一段 27.98 s 的正方形航线 (SQLite rosbag: 2799 条 /drone/odom)。图 3 是由 make_experiment_figures.py 从 CDR 消息直接解码得到的曲线。该次测试末状态为 (0.000,0.004,1.500) m，最大位置误差为 2.892 m；由于采用无规划器正方形航线且起点邻近显式障碍物，计算出的最小净空为 -0.593 m ，属于有意保留的失败/风险案例，而非安全绕障结论。

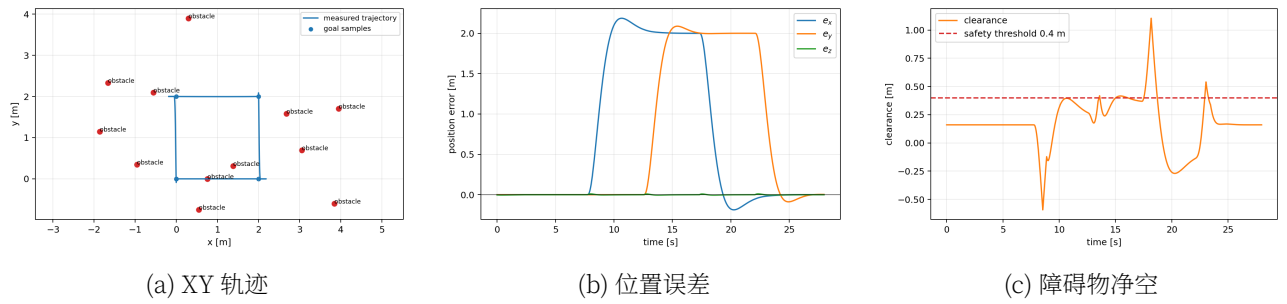


Figure 3: v1.1.0 分支独立复现实测（无规划器正方形航线）。

6.4 控制性能分析

阻尼特性 位置环 $\zeta \approx 0.7$ （二阶系统理论最优阻尼比），平衡响应速度与超调

积分死区 0.02 m 死区 + $\times 0.99$ /步衰减：小误差时积分逐渐收敛，防止 windup
 加速度限幅 $a_{xy} \leq 4 \text{ m/s}^2$, $a_z \in [-3, 6] \text{ m/s}^2$ ：保证推力分配不超出物理限制

6.5 失败场景

1. NO_PATH：障碍物完全阻塞，保留 last safe_goal，悬停。planner_status = NO_PATH，RViz 和地面站同步
2. 目标在障碍物内：A* 前 BFS 找最近自由格（半径 50），找不到则返回空路径 → 悬停
3. IMU 噪声 + 无滤波：bonus 场景—— $\sigma_a = 0.08$, $\sigma_g = 0.015$ 无滤波，控制器仍稳定收敛——验证 PID 鲁棒性

7 与参考仓库的关系

7.1 pengyu_sim (ROS1)

参考：四旋翼动力学公式和 mixer 推导。

重写：全部 ROS1 Python 改为 ROS2 C++ (rclcpp)。单线程 rospy.spin() 改为 MT executor + 隔离 callback group——1kHz dynamics 定时器不再饿死 RPM 订阅。pengyu_sim 无避障规划器、无传感器噪声、无风扰——这些全部从零设计。

7.2 MARSIM

参考：点云/障碍物感知思路。

重写：显式几何体 + 膨胀栅格取代点云。保留 3D 感知 (z 跨度检查、体素 FOV) 但避免点云管线复杂度。显式列表 ↔ 程序化随机双模式更适合验收。

7.3 设计原则

1. 全 C++ rclcpp：避开 conda Python 3.11 vs ROS2 3.10 冲突，任务也推荐核心用 C++
2. header-only drone_common：mixer B/B^{-1} 单信源——杜绝“符号错翻车”
3. MT executor + 隔离 callback group：1kHz dynamics 与订阅不互锁
4. 标准消息：一律使用 ROS2 标准类型，无需自定义.msg，构建拓扑最简
5. 可配性优先：YAML + ros2 param set 热更新（增益/风扰/噪声/感知模式/FOV）

8 AI 使用说明

8.1 使用的 AI 工具

整个工程实现阶段使用 Claude Code + DeepSeek-V4-Pro 完成协作式工程设计、代码生成、调试、单元测试与文档工作；本份报告则由 Codex + GPT-5.6-luna 完成整理、改写、排版和交付校验。AI 输出均经过人工运行验证，不将生成内容直接视为实验事实。

8.2 人与 AI 的分工

本人负责整体架构设计、核心公式物理核对、参数最终调整、验收端到端测试执行，以及报告内容的调整与最终确定。AI 主要负责实际工程设计的展开、代码生成、单元测试与文档初稿；涉及动力学符号、控制分配、障碍物布局和验收结论的部分由本人确认后才纳入工程。

8.3 AI 完成的模块

工作区骨架 (6 包)、drone_common 全部头文件 + 8 条 gtest、drone_dynamics (EOM + 电机 + 风扰 + IMU/Odom 噪声)、drone_controller (V2 线性 PD + Lee 姿态 + LADRC)、drone_map (显式/程序化)、drone_planner (grid/astar/astar3d/perception 四模式)、launch 文件 (3)、地面站 (HTTP + Chart.js + 控制面板)、全部辅助脚本、RViz 配置、README 与文档。

8.4 人类确认/修改的核心内容

Claude Code 调教记录显示，开发过程采用“先约束接口与验收标准，再迭代实现—构建—测试—修正”的方式：先固定 ROS 2 topic、频率和消息类型，再要求 AI 将动力学、控制、地图和规划拆成可独立验证的包；每次出现编译、运行或物理符号问题，都以终端日志和端到端现象作为下一轮修改依据。该过程避免了只根据代码阅读宣称通过。

- Mixer B 各行符号：自行推导叉乘，确认 τ_x (roll)、 τ_y (pitch)、 τ_z (yaw) 物理正确
- roll/yaw 符号断言：要求 gtest 加「+roll → 左电机快」「+yaw → CCW 电机快」物理断言
- 四元数积分：确认 $q \otimes \Omega$ 显式积分 + 每步归一化
- 碰撞感知平滑：要求 Bresenham 碰撞检查而非纯几何共线 + 坍塌 fallback
- PID 设计：要求学习参考代码的设计原则（分离、反饱和、死区），自行推导线性 PD + 0.02m 死区 + 遇限削弱
- 障碍物布局：自行设计 6 障碍物位置和窄通道场景

8.5 AI 产生的关键错误及修正

会话中还反复强调了“可运行证据优先”的约束：AI 不能用理想化曲线替代 rosbag/地面站记录，不能把 planner 的目标点误写成控制器实际接收的 safe_goal，也不能把一次编译成功扩大解释为整套验收通过。最终采用人工核对公式、人工选择参数、脚本化测试和视频复核的闭环。

- 1kHz timer 饿死订阅：单线程 spin 下 RPM 回调永不触发。→ MT executor(2)+ 独立 callback group
- 非线性 PID 超调 + 收敛慢：exp 型 P 公式实际偏差大。→ V2 简化为线性 PD + 死区积分
- RViz Enabled: true 破坏面板：侧栏消失。→ 恢复不含 Enabled 键的格式
- conda Python 破坏 CLI：→ 用系统 Python + LD_LIBRARY_PATH
- header-only 包无 imported target：→ INTERFACE library + ament_export_targets
- Assert 在 Release 关闭：→ 启动时显式参数校验 + cwiseMax(0) 硬件兜底

9 反思：如果继续做两周

9.1 最应该补的内容

1. 完整实验数据自动化 pipeline：当前手动 rosbag + 地面站 CSV。应建 CI——每 commit 触发 6 场景录制 + 自动出图 + 数值断言，形成可追溯性能回归基线
2. 多无人机协同避障：multi_drone.launch.py 仅命名空间隔离。可加 central planner 做 mutual avoidance
3. 单元测试扩展：仅 drone_common 有 gtest。dynamics/controller/planner 核心逻辑均应加入回归测试
4. 传感器与扰动下的稳态优化：开启 IMU 测量误差后，无人机跟踪的静差尚未完全优化；风扰动后的恢复速度也还可以进一步加快。

9.2 更深入的方向

1. 轨迹跟踪 → 微分平坦：waypoint-by-waypoint 切换有惯性弯曲。改用 minimum-snap 轨迹 + 前馈 (differential flatness)，精确跟踪光滑轨迹
2. LADRC 完善：当前独立三轴 LADRC 忽略内环耦合。可改为 cascaded LADRC + 控制量饱和和反馈到 LESO
3. 物理引擎对接：手写 EOM 对教学足够，但缺复杂接触/碰撞。可对接 Gazebo Ignition 或 MuJoCo
4. 机载计算延迟模拟：planner 加入 50-100ms 延迟，验证控制器在规划滞后下的鲁棒性
5. CI/CD：GitHub Actions 自动 colcon build + test，每 PR 跑全验收
6. 真实硬件部署：适配 PX4/ArduPilot 接口，在真机上验证控制参数
7. 泛化策略控制：如果有更多时间，希望引入论文 RAPTOR: A Foundation Policy for Quadrotor Control (Eschmann, Albani, Loianno, [arXiv:2509.11481](https://arxiv.org/abs/2509.11481)) 中的策略思路，在不同动力学、风扰和传感器条件下研究策略泛化与更好的控制性能，并与当前解析控制器做安全约束下的对比。